

1. Introduction to Hibernate Framework

Definition

- Hibernate is a powerful Object-Relational Mapping (ORM) framework for Java.
- It simplifies the interaction between Java applications and relational databases by allowing developers to work with Java objects instead of SQL queries.

Key Features

- **Object-Relational Mapping (ORM):**
 - Maps Java classes to database tables.
 - Maps Java data types to SQL data types.
 - **HQL (Hibernate Query Language):**
 - A powerful, SQL-like query language that operates on Java objects rather than database tables.
 - **Automatic Schema Generation:**
 - Hibernate can automatically create and update database schemas based on the entity classes.
 - **Caching:**
 - Supports first-level and second-level caching to improve performance.
 - **Transaction Management:**
 - Integrates with Java Transaction API (JTA) for managing transactions.
 - **Support for JPA:**
 - Implements the Java Persistence API (JPA) specification, allowing for portability across different JPA implementations.
-

2. Basic Configuration

Steps to Configure Hibernate

1. Entity Class:

- Define your entity class using JPA annotations.
- **Example:**

```
java
VerifyOpen In EditorRunCopy code
1package org.hibernate.example;
2
3import jakarta.persistence.Entity;
4import jakarta.persistence.Id;
5import jakarta.validation.constraints.NotNull;
6
7@Entity
8public class Book {
9    @Id
10    private String isbn;
11
12    @NotNull
```

```

13     private String title;
14
15     // Default constructor
16     public Book() {}
17
18     // Parameterized constructor
19     public Book(String isbn, String title) {
20         this.isbn = isbn;
21         this.title = title;
22     }
23
24     // Getters and Setters
25     public String getIsbn() {
26         return isbn;
27     }
28
29     public void setIsbn(String isbn) {
30         this.isbn = isbn;
31     }
32
33     public String getTitle() {
34         return title;
35     }
36
37     public void setTitle(String title) {
38         this.title = title;
39     }
40 }

```

2. Main Class:

- Configure and bootstrap Hibernate.
- **Example:**

```

java
VerifyOpen In EditorRunCopy code
1package org.hibernate.example;
2
3import org.hibernate.cfg.Configuration;
4import static org.hibernate.cfg.AvailableSettings.*;
5
6public class Main {
7    public static void main(String[] args) {
8        var sessionFactory = new Configuration()
9            .addAnnotatedClass(Book.class)
10            .setProperty(URL, "jdbc:h2:mem:db1") // In-memory
11            database
12                .setProperty(USER, "sa")
13                .setProperty(PASS, "")
14                .setProperty(SHOW_SQL, "true")
15                .setProperty(FORMAT_SQL, "true")
16                .buildSessionFactory();
17
18        // Persist an entity
19        sessionFactory.inTransaction(session -> {
20            session.persist(new Book("9781932394153", "Hibernate
in Action"));
21        });
22
23        // Query data using HQL
24        sessionFactory.inSession(session -> {
25            var result = session.createSelectionQuery("from Book",
Book.class).getResultList();

```

```

25         result.forEach(book ->
System.out.println(book.getTitle()));
26     });
27 }
28}

```

3. Gradle Build File:

- Include necessary dependencies in your `build.gradle`.
- **Example:**

```

groovy
VerifyOpen In EditorRunCopy code
1plugins {
2    id 'java'
3}
4
5group = 'org.example'
6version = '1.0-SNAPSHOT'
7
8repositories {
9    mavenCentral()
10}
11
12dependencies {
13    implementation 'org.hibernate.orm:hibernate-core:6.3.0.Final'
14    implementation 'org.hibernate.validator:hibernate-
validator:8.0.0.Final'
15    implementation 'com.h2database:h2:2.1.214' // H2 database
16}

```

3. Using Hibernate

Key Operations

- **Persisting Entities:**
 - Use the `session.persist()` method to save an entity.
- **Querying Data:**
 - Use HQL or Criteria API to retrieve data.
- **Transaction Management:**
 - Use `session.beginTransaction()` and `transaction.commit()` to manage transactions.

Example of Persisting and Querying

```

java
VerifyOpen In EditorRunCopy code
1// Persist an entity
2SessionFactory.inTransaction(session -> {
3    session.persist(new Book("9781932394153", "Hibernate in Action"));
4});
5
6// Query data using HQL
7SessionFactory.inSession(session -> {
8    var result = session.createQuery("from Book",
Book.class).getResultList();

```

```
9     result.forEach(book -> System.out.println(book.getTitle()));
10});
```

4. Hibernate Framework Architecture Overview

Core Components

1. Configuration:

- The `Configuration` class initializes Hibernate and reads configuration files (like `hibernate.cfg.xml`).

- **Example:**

```
java
VerifyOpen In EditorRunCopy code
1Configuration cfg = new Configuration();
2cfg.configure(); // Reads hibernate.cfg.xml
```

2. SessionFactory:

- A thread-safe, heavyweight object that creates `Session` instances.
- Typically created once during application startup.

- **Example:**

```
java
VerifyOpen In EditorRunCopy code
1SessionFactory factory = cfg.buildSessionFactory();
```

3. Session:

- A lightweight, single-threaded object that represents a conversation between the application and the database.
- Used to perform CRUD operations and manage the persistence context.

- **Example:**

```
java
VerifyOpen In EditorRunCopy code
1Session session = factory.openSession();
```

4. Transaction:

- The `Transaction` interface is used to manage database transactions.
- Allows you to begin, commit, and rollback transactions.

- **Example:**

```
java
VerifyOpen In EditorRunCopy code
1Transaction tx = session.beginTransaction();
2// Perform operations
3tx.commit(); // Make changes permanent
```

5. Query and Criteria:

- Hibernate provides **Query** and **Criteria** interfaces to retrieve data from the database.
- **Example for Query:**

```
java
VerifyOpen In EditorRunCopy code
1Query query = session.createQuery("FROM EntityName");
2List results = query.list();
```

Flow of Operations in Hibernate

1. **Initialization:**
 - The application initializes Hibernate by creating a **Configuration** object and building a **SessionFactory**.
 2. **Session Management:**
 - A **Session** is opened from the **SessionFactory** whenever a database operation is needed.
 3. **Transaction Handling:**
 - A transaction is started using the **Session** object. All operations performed within this transaction can be committed or rolled back.
 4. **CRUD Operations:**
 - The application performs Create, Read, Update, and Delete operations using the **Session** object.
 5. **Closing Resources:**
 - After operations are complete, the **Session** is closed to free up resources.
-

5. Instance States in Hibernate

States of Instances

- **Transient:**
 - The instance is not associated with any session.
 - Changes made to the entity do not affect the database.
- **Persistent:**
 - The instance is associated with a session and can be saved to the database.
 - Hibernate automatically detects changes and updates the database accordingly.
- **Detached:**
 - The instance was once associated with a session but is now disconnected.
 - Changes made to the entity will not be automatically synchronized with the database.
- **Removed:**
 - The instance enters the removed state when it is deleted from the database using the `delete()` method.
 - The entity is marked for removal, and the actual deletion occurs during the next flush operation.

Summary of Lifecycle States

- **Transient:** Not associated with any session; changes do not affect the database.
 - **Persistent:** Associated with a session; changes are tracked and synchronized with the database.
 - **Detached:** No longer associated with a session; changes are not synchronized unless reattached.
 - **Removed:** Marked for deletion; will be removed from the database upon transaction commit.
-

6. Creating a Web Application Using Hibernate API

Steps to Create a Web Application

1. Set Up Your Development Environment:

- Use an IDE like Eclipse or IntelliJ IDEA.
- Ensure you have Java Development Kit (JDK) installed.
- Set up a web server (like Apache Tomcat) for deploying your application.
- Use Maven for dependency management.

2. Create a Maven Project:

- Create a new Maven project in your IDE.
- Update the `pom.xml` file to include necessary dependencies for Hibernate, MySQL, and any other libraries you may need.
- **Example:**

```
xml
VerifyOpen In EditorRunCopy code
1<project xmlns="http://maven.apache.org/POM/4.0.0"
2      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
4  <modelVersion>4.0.0</modelVersion>
5  <groupId>com.example</groupId>
6  <artifactId>HibernateWebApp</artifactId>
7  <version>1.0-SNAPSHOT</version>
8  <dependencies>
9      <dependency>
10         <groupId>org.hibernate</groupId>
11         <artifactId>hibernate-core</artifactId>
12         <version>5.4.30.Final</version>
13     </dependency>
14     <dependency>
15         <groupId>mysql</groupId>
16         <artifactId>mysql-connector-java</artifactId>
17         <version>8.0.23</version>
18     </dependency>
19     <dependency>
20         <groupId>javax.servlet</groupId>
21         <artifactId>javax.servlet-api</artifactId>
22         <version>4.0.1</version>
23         <scope>provided</scope>
24     </dependency>
25 </dependencies>
```

26</project>

3. Create Hibernate Configuration File:

- Create a file named `hibernate.cfg.xml` in the `src/main/resources` directory.

- **Example:**

```
xml
VerifyOpen In EditorRunCopy code
1<?xml version="1.0" encoding="utf-8"?>
2<!DOCTYPE hibernate-configuration PUBLIC
3    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4    "http://hibernate.sourceforge.net/hibernate-configuration-
3.0.dtd">
5<hibernate-configuration>
6    <session-factory>
7        <property
name="hibernate.dialect">org.hibernate.dialect.MySQL5Dialect</proper
ty>
8        <property
name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</
property>
9        <property
name="hibernate.connection.url">jdbc:mysql://localhost:3306/your_dat
abase</property>
10        <property
name="hibernate.connection.username">your_username</property>
11        <property
name="hibernate.connection.password">your_password</property>
12        <property name="hibernate.hbm2ddl.auto">update</property>
13        <property name="show_sql">true</property>
14    </session-factory>
15</hibernate-configuration>
```

4. Create Entity Classes:

- Create a Java class that represents the entity you want to persist.

- **Example:**

```
java
VerifyOpen In EditorRunCopy code
1package com.example.model;
2
3import javax.persistence.*;
4import java.util.Date;
5
6@Entity
7@Table(name = "employee")
8public class Employee {
9    @Id
10    @GeneratedValue(strategy = GenerationType.IDENTITY)
11    private Long id;
12
13    private String name;
14    private String role;
15
16    @Temporal(TemporalType.TIMESTAMP)
17    private Date insertTime;
18
19    // Getters and Setters
20}
```

5. Create Hibernate Utility Class:

- Create a utility class to manage the Hibernate SessionFactory.
- **Example:**

```
java
VerifyOpen In EditorRunCopy code
1package com.example.util;
2
3import org.hibernate.SessionFactory;
4import org.hibernate.cfg.Configuration;
5
6public class HibernateUtil {
7    private static final SessionFactory sessionFactory =
buildSessionFactory();
8
9    private static SessionFactory buildSessionFactory() {
10        try {
11            return new
Configuration().configure().buildSessionFactory();
12        } catch (Throwable ex) {
13            throw new ExceptionInInitializerError(ex);
14        }
15    }
16
17    public static SessionFactory getSessionFactory() {
18        return sessionFactory;
19    }
20
21    public static void shutdown() {
22        getSessionFactory().close();
23    }
24}
```

6. Create a Servlet to Handle Requests:

- Create a servlet that will handle HTTP requests and interact with Hibernate.
- **Example:**

```
java
VerifyOpen In EditorRunCopy code
1package com.example.servlet;
2
3import com.example.model.Employee;
4import com.example.util.HibernateUtil;
5import org.hibernate.Session;
6import org.hibernate.Transaction;
7
8import javax.servlet.ServletException;
9import javax.servlet.annotation.WebServlet;
10import javax.servlet.http.HttpServlet;
11import javax.servlet.http.HttpServletRequest;
12import javax.servlet.http.HttpServletResponse;
13import java.io.IOException;
14import java.util.Date;
15
16@WebServlet("/addEmployee")
17public class EmployeeServlet extends HttpServlet {
18    protected void doPost(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException {
19        String name = request.getParameter("name");
20        String role = request.getParameter("role");
```



```

21
22     Employee employee = new Employee();
23     employee.setName(name);
24     employee.setRole(role);
25     employee.setInsertTime(new Date());
26
27     Transaction transaction = null;
28     try (Session session =
HibernateUtil.getSessionFactory().openSession()) {
29         transaction = session.beginTransaction();
30         session.save(employee);
31         transaction.commit();
32     } catch (Exception e) {
33         if (transaction != null) transaction.rollback();
34         e.printStackTrace();
35     }
36
37     response.sendRedirect("success.jsp");
38 }
39}

```

7. Create JSP Pages for User Interface:

- Create a simple JSP page to collect employee data.
- **Example:**

```

jsp
VerifyOpen In EditorRunCopy code
1<%@ page contentType="text/html; charset=UTF-8" language="java" %>
2<html>
3<head>
4    <title>Add Employee</title>
5</head>
6<body>
7    <h2>Add Employee</h2>
8    <form action="addEmployee" method="post">
9        Name: <input type="text" name="name" required><br>
10       Role: <input type="text" name="role" required><br>
11       <input type="submit" value="Add Employee">
12    </form>
13</body>
14</html>

```

8. Deploy and Run the Application:

- Package your application as a WAR file and deploy it to your Tomcat server.
- Access the application through your web browser.

7. Hibernate Entity Lifecycle Explanation

Lifecycle States

- **Transient State:**
 - An entity is in the transient state when it is created but not yet associated with a Hibernate session.
- **Characteristics:**
 - The entity exists only in memory.

- Changes made to the entity do not affect the database.
- **Persistent State:**
 - An entity enters the persistent state when it is associated with a Hibernate session.
 - **Characteristics:**
 - The entity is tracked by the session.
 - Hibernate automatically detects changes and updates the database accordingly.
- **Detached State:**
 - An entity becomes detached when the session it was associated with is closed or when the entity is explicitly detached.
 - **Characteristics:**
 - Changes made to the entity will not be automatically synchronized with the database.
 - The entity can be reattached to a new session.
- **Removed State:**
 - An entity enters the removed state when it is deleted from the database using the `delete()` method.
 - **Characteristics:**
 - The entity is no longer present in the database after the transaction is committed.
 - Any changes made to the entity after it has been removed will not affect the database.

Summary of Lifecycle States

- **Transient:** Not associated with any session; changes do not affect the database.
 - **Persistent:** Associated with a session; changes are tracked and synchronized with the database.
 - **Detached:** No longer associated with a session; changes are not synchronized unless reattached.
 - **Removed:** Marked for deletion; will be removed from the database upon transaction commit.
-

8. Hibernate Annotations Example

Using Annotations

- Hibernate annotations provide a way to define the mapping between Java classes and database tables directly in the Java code, eliminating the need for XML configuration files.
- **Example:**

```

java
VerifyOpen In EditorRunCopy code
1package com.example.model;
2
3import jakarta.persistence.*;
4
5@Entity

```

```

6@Table(name = "employee")
7public class Employee {
8    @Id
9    @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private Long id;
11
12    @Column(name = "name", nullable = false)
13    private String name;
14
15    @Column(name = "role", nullable = false)
16    private String role;
17
18    // Constructors, Getters, and Setters
19}

```

9. Hibernate Mappings and Relationships Examples

Types of Relationships

1. One-to-One Mapping:

- A one-to-one relationship means that one entity is associated with exactly one instance of another entity.
- **Example:** Employee and Account.
- **Code Example:**

```

java
VerifyOpen In EditorRunCopy code
1@Entity
2public class EmployeeEntity {
3    @Id
4    @GeneratedValue(strategy = GenerationType.IDENTITY)
5    private Integer employeeId;
6
7    @OneToOne(mappedBy = "employee", cascade = CascadeType.ALL)
8    private AccountEntity account;
9}
10
11@Entity
12public class AccountEntity {
13    @Id
14    @GeneratedValue(strategy = GenerationType.IDENTITY)
15    private Integer accountId;
16
17    @OneToOne
18    @JoinColumn(name = "employee_id")
19    private EmployeeEntity employee;
20}

```

2. One-to-Many and Many-to-One Mapping:

- In a one-to-many relationship, one entity can be associated with multiple instances of another entity.
- **Example:** Department and Employees.
- **Code Example:**

```

java

```

```

VerifyOpen In EditorRunCopy code
1@Entity
2public class DepartmentEntity {
3    @Id
4    @GeneratedValue(strategy = GenerationType.IDENTITY)
5    private Integer departmentId;
6
7    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL)
8    private List<EmployeeEntity> employees;
9}
10
11@Entity
12public class EmployeeEntity {
13    @ManyToOne
14    @JoinColumn(name = "department_id")
15    private DepartmentEntity department;
16}

```

3. Many-to-Many Mapping:

- In a many-to-many relationship, multiple instances of one entity can be associated with multiple instances of another entity.
- **Example:** Students and Courses.
- **Code Example:**

```

java
VerifyOpen In EditorRunCopy code
1@Entity
2public class StudentEntity {
3    @Id
4    @GeneratedValue(strategy = GenerationType.IDENTITY)
5    private Integer studentId;
6
7    @ManyToMany(cascade = CascadeType.ALL)
8    @JoinTable(
9        name = "Student_Course",
10        joinColumns = @JoinColumn(name = "student_id"),
11        inverseJoinColumns = @JoinColumn(name = "course_id")
12    )
13    private Set<CourseEntity> courses;
14}
15
16@Entity
17public class CourseEntity {
18    @Id
19    @GeneratedValue(strategy = GenerationType.IDENTITY)
20    private Integer courseId;
21
22    @ManyToMany(mappedBy = "courses")
23    private Set<StudentEntity> students;
24}

```

10. Hibernate Collection and Component Mapping

Collection Mapping

- Hibernate supports various types of collections, including Set, List, Map, and Bag.
- **Example:** One-to-Many relationship between Department and Employees.

- **Code Example:**

```
java
VerifyOpen In EditorRunCopy code
1@Entity
2public class Department {
3    @Id
4    @GeneratedValue(strategy = GenerationType.IDENTITY)
5    private Long id;
6
7    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL)
8    private Set<Employee> employees;
9}
10
11@Entity
12public class Employee {
13    @Id
14    @GeneratedValue(strategy = GenerationType.IDENTITY)
15    private Long id;
16
17    @ManyToOne
18    @JoinColumn(name = "department_id")
19    private Department department;
20}
```

Component Mapping

- Component mapping allows you to map a class as a value type, meaning it does not have its own identity and is part of another entity.
- **Example:** Address as a component of User.
- **Code Example:**

```
java
VerifyOpen In EditorRunCopy code
1@Entity
2public class User {
3    @Id
4    @GeneratedValue(strategy = GenerationType.IDENTITY)
5    private Long id;
6
7    @Embedded
8    private Address address;
9}
10
11@Embeddable
12public class Address {
13    private String street;
14    private String city;
15}
```

11. Hibernate HQL (Hibernate Query Language) with Examples

Key Features of HQL

- **Object-Oriented:** Works with Java objects and their properties.

- **Case Insensitivity:** HQL keywords are case-insensitive.
- **Supports Joins:** Allows for complex queries involving multiple entities.
- **Aggregate Functions:** Supports functions like `count()`, `sum()`, `avg()`, etc.

Basic HQL Syntax

- `FROM EntityName`
- `SELECT property1, property2 FROM EntityName`
- `WHERE condition`
- `ORDER BY property`

Examples of HQL Queries

1. Retrieving All Records:

```
java
VerifyOpen In EditorRunCopy code
1String hql = "FROM Employee"; // Retrieves all Employee records
2Query query = session.createQuery(hql);
3List<Employee> employees = query.list();
```

2. Retrieving Specific Columns:

```
java
VerifyOpen In EditorRunCopy code
1String hql = "SELECT e.name, e.role FROM Employee e"; // Retrieves names
and roles of all employees
```

3. Using WHERE Clause:

```
java
VerifyOpen In EditorRunCopy code
1String hql = "FROM Employee e WHERE e.role = :role"; // Retrieves
employees with a specific role
2query.setParameter("role", "Developer");
```

4. Using ORDER BY Clause:

```
java
VerifyOpen In EditorRunCopy code
1String hql = "FROM Employee e ORDER BY e.name ASC"; // Retrieves all
employees sorted by name
```

5. Using Aggregate Functions:

```
java
VerifyOpen In EditorRunCopy code
1String hql = "SELECT COUNT(e) FROM Employee e"; // Counts the number of
employees
```

6. Using Joins:

```
java
VerifyOpen In EditorRunCopy code
1String hql = "SELECT e.name, a.city FROM Employee e INNER JOIN e.address
a"; // Retrieves employee names and their associated cities
```

7. Updating Records:

```
java
VerifyOpen In EditorRunCopy code
1String hql = "UPDATE Employee e SET e.role = :newRole WHERE e.id
= :id"; // Updates the role of a specific employee
```

8. Deleting Records:

```
java
VerifyOpen In EditorRunCopy code
1String hql = "DELETE FROM Employee e WHERE e.id = :id"; // Deletes a
specific employee
```

12. Hibernate Named Queries with Examples

Definition

- Hibernate Named Queries are a powerful feature that allows you to define queries with a specific name, which can be reused throughout your application.

Defining Named Queries

- You can define Named Queries using the `@NamedQuery` and `@NamedNativeQuery` annotations in your entity classes.
- **Example:**

```
java
VerifyOpen In EditorRunCopy code
1@Entity
2@NamedQueries({
3    @NamedQuery(name = "Employee.findAll", query = "FROM Employee"),
4    @NamedQuery(name = "Employee.findById", query = "FROM Employee e
WHERE e.id = :id")
5})
6public class Employee {
7    // Fields, Getters, and Setters
8}
```

Using Named Queries

- You can use the defined Named Queries in your application code.
- **Example:**

```
java
VerifyOpen In EditorRunCopy code
1List<Employee> employees = session.createNamedQuery("Employee.findAll",
Employee.class).getResultList();
```

13. Hibernate Criteria Queries with Examples

Criteria API

- Hibernate's Criteria API provides a powerful and flexible way to create queries programmatically.
- It allows developers to build queries using Java objects rather than SQL strings.

Examples of Criteria Queries

1. Basic Criteria Query:

```
java
VerifyOpen In EditorRunCopy code
1Criteria criteria = session.createCriteria(Employee.class);
2List<Employee> employees = criteria.list();
```

2. Adding Restrictions:

```
java
VerifyOpen In EditorRunCopy code
1criteria.add(Restrictions.eq("role", "Developer")); // Filter by role
```

3. Using Projections:

```
java
VerifyOpen In EditorRunCopy code
1criteria.setProjection(Projections.rowCount()); // Count total employees
```

4. Ordering Results:

```
java
VerifyOpen In EditorRunCopy code
1criteria.addOrder(Order.asc("name")); // Order by name ascending
```

5. Combining Restrictions:

```
java
VerifyOpen In EditorRunCopy code
1Disjunction disjunction = Restrictions.disjunction();
2disjunction.add(Restrictions.eq("role", "Developer"));
3disjunction.add(Restrictions.eq("role", "Manager"));
4criteria.add(disjunction); // Employees who are either Developer or Manager
```

6. Using Subqueries:

```
java
VerifyOpen In EditorRunCopy code
1DetachedCriteria subquery = DetachedCriteria.forClass(Department.class)
2    .add(Restrictions.eq("name", "IT"))
3    .setProjection(Property.forName("id"));
4criteria.add(Property.forName("department.id").in(subquery)); // Employees in IT department
```

This structured overview provides a comprehensive understanding of the Hibernate framework, its configuration, usage, architecture, lifecycle, and various features. Each section is designed to help

you grasp the concepts effectively, with examples to illustrate practical applications. If you have any specific questions or need further elaboration on any topic, feel free to ask!